

SE

06.10.25

Why?

- Instandhaltung des Zwingers der Komponenten
- Sicherheitsgarantie durch Risikoabschätzung und Halten an Standards und Gesetze, Dokumentation
- Kostenschätzung (Einhalten der Vorgaben und Pläne)

MEH Kosten?
Matrix Org?

What?

- Planmäßige Entw. eines Sys (nach Standards, (Vorgaben, ...))
("nach Methode")
- Requirements - Kommunikation
- Interfaces in vielen Ebenen
- SE auch später, bspw. Wartung, neue Releases etc.

Wozu?

- Systemgrenzen definieren zugleich Einteilung der Verantwortungen, i.e. wer ist zuständig für was? → Planung, Scope
- Emergenz bezeichnen: tritt hervor durch das Zusammenspiel von vielen Systemen
→ emergent properties: only arise from the system's interconnection

Wann ist smth complex?

- depends on personal knowledge and understanding of the object

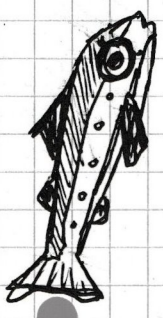
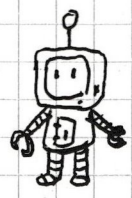
Komplexität: Genauigkeit, Detail, Umfang ↔ Abhängigkeit

Complicated Systems

- "... is difficult to ~~not~~ be overlooked..."
- "... interaction between the many parts are governed by fixed relationships..."
- Verständnis kommt durch Zerlegung des Systems
- "... better described as a multilayered system ..."
- complexity arises through ^{dynamic} ~~static~~ changes and its processes?

Dynamic vs static

- no one clear definition
- unordered system/set: sum of its parts
- emergence: connected parts create a global function/system



SE

contd.
06.10.25

5 key properties of a complex system:

- consist of many parts
- hierarchical structure: nesting of system create bigger system, interconnected and interdependent
- local interaction and self-organisation

~~Transfer to a complex technical system:~~

- nonlinearity: greater or less than its parts, its sum is not often as expected
- complex systems can change rapidly → sensitivity to initial conditions and tiny changes
- connectivity: high level of interconnectivity which defines the system instead of its parts, flow is more important
- autonomy and adaptation: reactivity to its surrounding environments
- no centralized organisation → subject to evolution (meaning that natural selection determines how the sys. evolves)

Summary:

1. Numerosity 2. Nonlinearity 3. Connectivity 4. Adaptation

- Anzahl der Elemente
- Emergenz
- Autonomie d. Elemente
- nicht linear: unerwartetes Ergebnis

Ex.: Games as complex systems

- depending on scope many interconnected elements that create the gameplay
- each subsystem (inventory, items, UI) has a degree of autonomy and self-organisation to regulate and organise itself as well as adapt to its surrounding system
- depending on the game, the game world is dynamic and changes in accordance with itself and player input

SE

13.10.25

Ex.:
1.2 Open



→ Numerosity:

SysE

Contol.
17.11.25

~~Kritik~~ **Review** - Brainstorming in viele Richtungen

- iterativ → Idee → Feedback

- schlechte Idee (durch eigenes Empfinden und Bias) aber in Augen eines anderen actually eine gute Idee

→ Einschätzen ob gut oder schlecht durch User

- Lösung mit wenig Infos

- wenig Zeit → trotzdem Lösungsidee

- vielseitige Betrachtung des Problems und dessen Lösungen durch Constraints

- Stokes → Blind Contour Drawing or similar creates connection / ice breaker

Tools	
- Stokes	- Reframe offene Frage
- Why? Why	- Remix
- Storyboard	
- Mirror Map / Journey Map	
- Sketching with Constraints	
- Prototyping with Pro/Con Feedback	

Prüfung → Transfer Fragen : kleine Aufg. mit Fragestellung "Wie könnte man das angehen?"
→ Systemstreuung

→ Product Life Cycle (Verständnis abfrage)

↳ Übergang

↳ Rollen / Aufgaben von Systemingenieur

↳ eingebettet in Situation
→ Analyse

↳ Zerlegung in Komponente und deren ~~Bezieh~~ Beziehung
→ Design Matrix

- warum benutzt man bestimmte Methoden

- System Charakteristiken

→ Grenzen?

→ technisch?

→ komplex / kompliziert?

→ Design Thinking : Erfahrung, Tools, Einsatz Situation,

Definition, Prozess, Phasen

→ Transfer: was macht das für ein System-ingenieur ~~aus~~ was?

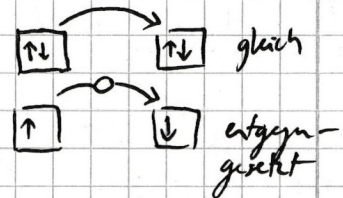
→ System Dynamics: Vorgabe Modell

↳ Analyse der Geops

↳ eigenes beschreiben

Standard: →

- wenn eigenes Standard benutzen dann mit Legende



24.11.25

Business or Mission Analysis ①

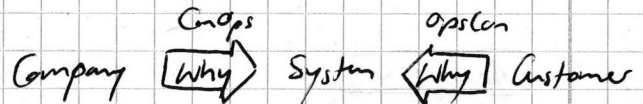
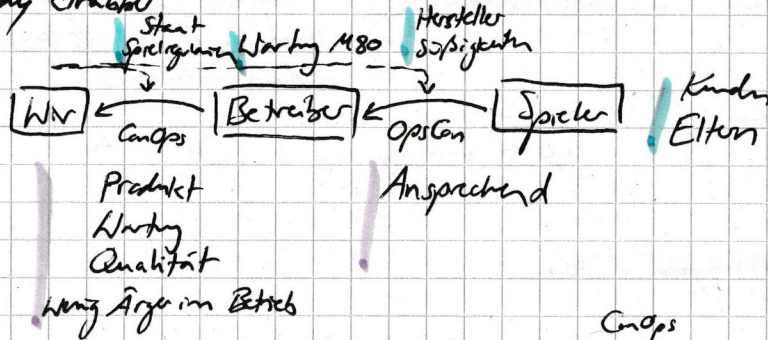
ConOps: Concept of Operation

intended way of business operation → how do we make money

OpsCon: System Operational Concept

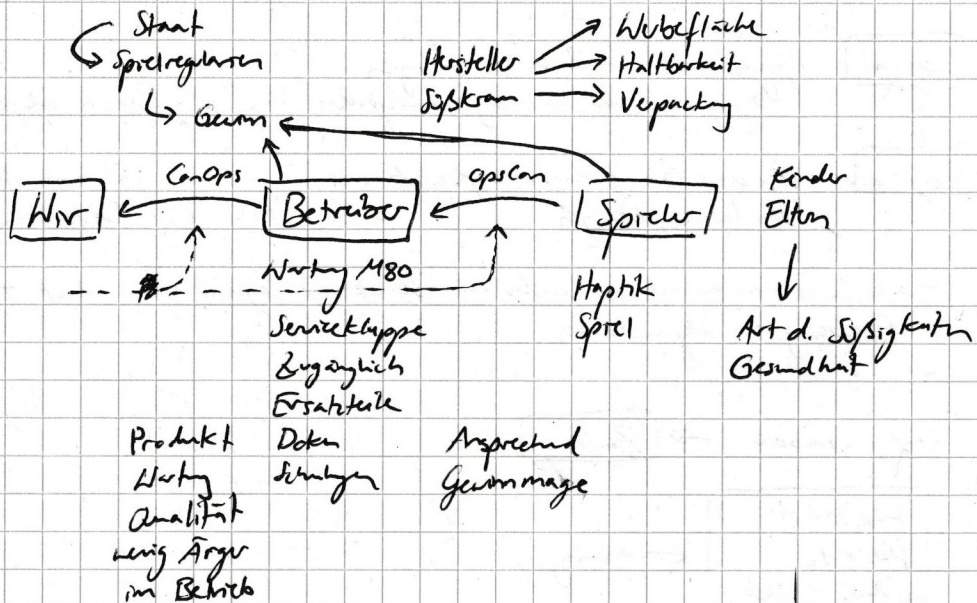
from a user's perspective → why do I need this?

Bsp.: Candy Grabber



Stakeholder Needs ②

- define stakeholder needs, derive requirements and organization, development of lifecycle concepts



System Requirements Definition Process ③

Architecture Definition Process ④

- architecture alternatives are compared to find most suitable for customer requirements and project constraints
- development of models for each alternative consisting of different views

Design Definition Process ⑤

- ~~ext~~ extension of found architecture into design by adding info to the model until implementable
- evaluation of alternative affirm or cancel chosen architecture

SysE

Contd.

24.11.25

System Analysis Process ⑥

- LCC (lifetime costs)
- Risk analysis: technical risks, ~~per~~ probabilities and impact
- Effectiveness analysis: meeting of operational properties throughout lifetime

Implementation and Integration Process ⑦

- ~~Verification~~ Verification Process ⑧ objective evidence that a system (element) fulfills ^{the} specified requirements and characteristics

Transition Process ⑨ - transfer of system custody from development team to customer

~~System~~ Analysis Process

SysE

01/12/25

Architecture

Base Operation Concepts / Zweck / Business Case
Domains (nötige Expertise finden)

Diagramm

REQ

BDD

Functional Funktionsdefinition Usecase
Funktionsbeschreibung

UC + Actor

Activity Diagram

WAS?



Logical Systemdeute und deren Verbindungen
Möglichkeit der Umsetzung identifizieren "dass"

Block Definition

Physical Wie das Ganze dann umgesetzt wird
Hardware, Design

"wie"

Internal Block
Definition

WIE?

- Use Cases aus Functional werden in Block von Logical eingesetzt →
Feststellung von Dependencies

Only Grabber → Zustand

Füllstand
Pos xyz
~~800~~ Geld
Pos Strk ABC

← abhängig

← unabhängig (können frei in system verwendet werden)

1 Leckerli
10 10 10
1 €
000

—
—
1 €
—

SysE

08/12/25

Dependent and independent variables of the Microwave Oven

- Start_Button_Pressed I
- Lid_Opened I
- Heat_On D
- Error_State D
- Lid_Status_Open D



Vor definieren, Zustandsübergänge definieren $\rightarrow$ Modellchecker

Designed Operations

- When Lid is closed and start button is pressed, heat is applied.

Constraints?

- Heat is off, when lid is open.
- From all states can return from all states back to start

Linear Temporal Logic LTL

$X \varphi$: φ is true in next state

$F \varphi$: φ is true at least once in the future

$G \varphi$: φ is true for all states

$\varphi_1 \text{ U } \varphi_2$: φ_1 is true until φ_2 is true

G Von allen Zuständen aus
 F gibt es einen zukünftigen Zustand
wo gilt lidopen $\wedge$ $\neg$ heat

~~$G \neg$ (lidopen $\wedge$ heat)~~

~~LTL SPEC F lidopen $\wedge$ heat~~

LTL SPEC $G F$ lidopen $\wedge$ $\neg$ heat

LTL SPEC $G \neg$ (lidopen $\wedge$ heat)

LTL SPEC $G F$ (no heat $\wedge$ door closed $\wedge$ no error $\wedge$ no start)

CTL - Computation Tree Logic

A Alle Wege

G gilt immer

E Es gibt einen Pfad

F auf dem gegebenen ist irgendwas

$G F \varphi$ (can start whenever due to G (all paths))

Ex 1

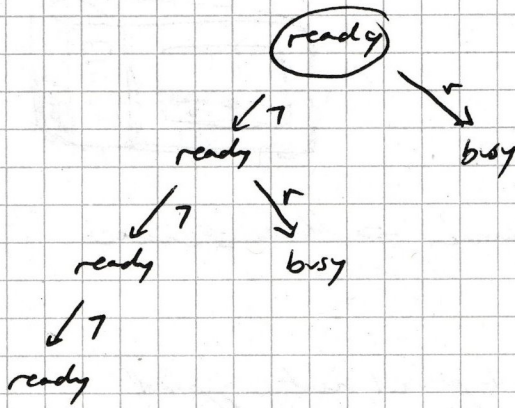
could

08/12/25

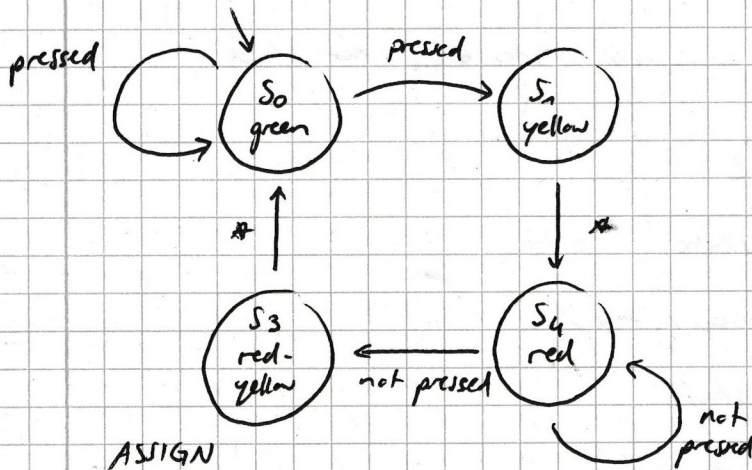
Finite State Machines in NUSMV

Additional

computation tree



Ex 01: Traffic Light



Ampel/rot rot/gelb gelb grün abhängig

Drücker = 1/0 unabhängig

FG

Fußgänger = rot grün abhängig

ASSIGN

init (Ampel) = grün

~~init (FG) = rot~~

next (Ampel) = CASE

drücker = FALSE: grün;

drücker = TRUE: gelb;

ampel = gelb: rot;

FG = rot: rotgelb;

ampel = rotgelb: grün;
TRUE: grün;
ESAC;

init (FG) = rot

next (FG) = CASE

ampel = rot: grün;

FG = grün: rot;

TRUE: rot;

ESAC;

INVAR SPEC $G((fg = grün \ \& \ ampel = grün))$

(does not work properly like this, was fixed in lecture)

□ Turnstile Ex

Asynchronous Systems

Synchron

S1

S2

- wechselt unregelmäßig/unfallig zwischen S1 und S2

S1 S1 S2 S2 S1 S2 S1 S1

Asynchron

next (Controll) = {S1, S2} (+ env für Environment)

S1

S2

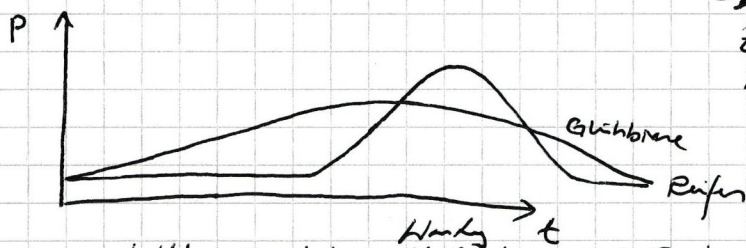
- durch Controll/Scheduler Kontrolle über Abfolge und Möglichkeit Environment mit einbeziehen

S1 S2 S1 S2 S1 S2 S1 S2

- FAIRNESS: alle states im Scheduler werden irgendwann mindestens einmal erreicht

Reliability Theory

Failure Distribution



- wie viele Monate / Zeitdauer bis Versagen der Komponente

→ hilft präventative Maßnahmen und Zeitpunkt zum Austausch festzustellen

- Run-to-failure Gleichbrunn kann schlecht gewartet werden, aufgrund der hohen Varianz

> Wartungsstrategie hängt Risiko auf Hinsicht des Systems ab, sowie die Art und Weise des Versagens.

Component Classification

Central?

- component classification: if the component fails, when do you notice it?
> if not immediately noticeable, then it is not a central component